# Correção · Tipos de teste · padrão FGV · 15 questões

**Data:** 04/11/2026 · **Semana 2** · **Placar: 8/15 (53%)** · 3 erros · 4 com "?" (não sabia)

**Respostas lidas das anotações do PDF:** 1-C · 2-A · 3-? · 4-? · 5-? · 6-A · 7-C · 8-A · 9-A · 10-D · 11-B · 12-A · 13-? · 14-A · 15-C

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.
>
> **Como li as marcas:** a tabela "Minhas respostas" ficou vazia, então usei os círculos nas letras. Nas Q3, Q4 e Q5 há um "?" grande ao lado das alternativas, sem letra marcada. Na Q13 o "?" está ao lado de "teste estático", que você circulou: não sabia o conceito. Na Q1 há pontinho ao lado da (C), a circulada, e a (B) ficou sem riscar, então conta como **acerto com dúvida**. Na Q6 há pontinho ao lado da (A). Na Q7 você desenhou o grafo e contou 4. Na Q12 você circulou "incorreta", marcou ✓ em (B) e (C) e circulou a (A). Na Q15 riscou a (A) com X e pôs pontinho na (B) antes de circular a (C).

| Questão | Assunto | Sua | Gabarito | Resultado |
|---|---|---|---|---|
| 1 | Níveis de teste (I a IV) | C | C | ✅ (com dúvida) |
| 2 | Alfa × beta | A | A | ✅ |
| 3 | Valor-limite | ? | D | ⬜ não sabia (?) |
| 4 | Partição de equivalência | ? | B | ⬜ não sabia (?) |
| 5 | Caixa-branca × caixa-preta | ? | E | ⬜ não sabia (?) |
| 6 | 💻 Cobertura de comandos × decisões | A | B | ❌ |
| 7 | 💻 Complexidade ciclomática | C | C | ✅ |
| 8 | Carga × estresse | A | A | ✅ |
| 9 | Confirmação × regressão | A | E | ❌ |
| 10 | Erro → defeito → falha | D | D | ✅ |
| 11 | Stub × driver (top-down) | B | B | ✅ |
| 12 | Princípios do teste ("incorreta") | A | D | ❌ |
| 13 | Teste estático × dinâmico | ? | E | ⬜ não sabia (?) |
| 14 | Pirâmide e automação | A | A | ✅ |
| 15 | 💻 TDD com stub | C | C | ✅ |

**Por tipo:**

| Tipo | Questões | Placar |
|---|---|---|
| Níveis, alfa/beta, stub × driver | 1, 2, 11 | 3/3 |
| Tipos de teste (carga, regressão) e erro/defeito/falha | 8, 9, 10 | 2/3 |
| **Técnicas** (valor-limite, equivalência, caixa-branca/preta, cobertura) | 3, 4, 5, 6 | **0/4** |
| Leitura de código | 6, 7, 15 | 2/3 |
| Princípios e estático × dinâmico | 12, 13 | 0/2 |
| Automação, pirâmide e TDD | 14, 15 | 2/2 |

**Comparação:** primeiro bloco só de testes. Antes, o assunto tinha aparecido de lado: modelo em V (erro em 26/10, acerto em 28/10) e TDD (acertos em 24/10 e 26/10). **Os níveis, os tipos e a automação estão bons.** O buraco está nas **técnicas de projeto de casos de teste** (valor-limite, partição, caixa-branca × preta e

cobertura): 0/4, três delas com "?". São justamente as que a FGV mais transforma em conta. A boa notícia é que são regras mecânicas: depois de entendidas, não se esquecem.

---

## Questão 1 · ✅ Acertou (com dúvida)

A equipe de TI de um Tribunal de Justiça está concluindo um novo módulo de distribuição de processos. Durante o projeto, foram realizadas as seguintes atividades de teste:

I. o desenvolvedor verificou, de forma isolada, o método que calcula o prazo de redistribuição, substituindo o acesso ao banco de dados por um objeto simulado;

II. a equipe verificou se o módulo de distribuição envia corretamente os dados do processo ao módulo de intimações, por meio da API interna;

III. a equipe de qualidade executou, em ambiente de homologação semelhante ao de produção, testes do sistema completo, incluindo os requisitos de tempo de resposta;

IV. servidores das varas, usuários finais do sistema, verificaram se ele atende às necessidades do trabalho cartorário antes da implantação.

As atividades I, II, III e IV correspondem, respectivamente, aos níveis de teste:

(A) integração, unitário, aceitação e sistema.

(B) unitário, sistema, integração e aceitação.

✅ **(C) unitário, integração, sistema e aceitação.** ← sua resposta

(D) unitário, integração, aceitação e sistema.

(E) integração, integração, sistema e regressão.

Você grifou as pistas certas ("de forma isolada", "sistema completo") e riscou A, D e E. A (B) ficou sem riscar. Pista para eliminá-la: o item II fala de **dois módulos conversando pela API**, e conversa entre módulos é integração. A (D) troca III e IV: quem verifica a **necessidade do usuário** é a aceitação, feita pelo próprio usuário.

📌 **Unitário** = isolado, pelo dev · **integração** = interface entre módulos · **sistema** = sistema completo, pela equipe de QA, inclui RNF · **aceitação** = usuário final valida a necessidade.

---

## Questão 2 · ✅ Acertou

Um Tribunal de Justiça desenvolveu um aplicativo para que advogados consultem a pauta de audiências. Antes do lançamento, o aplicativo passou por duas etapas de teste com usuários:

- na primeira, servidores convidados foram à sede da Secretaria de TI e usaram o aplicativo sob a observação dos desenvolvedores, que anotavam as dificuldades encontradas;
- na segunda, uma versão de pré-lançamento foi disponibilizada a um grupo de advogados, que a utilizaram em seus próprios escritórios, sem acompanhamento da equipe, e reportaram os problemas por um formulário.

As duas etapas descritas correspondem, respectivamente, a:

✅ **(A) teste alfa e teste beta.** ← sua resposta

(B) teste beta e teste alfa.

(C) teste de sistema e teste de aceitação contratual.

(D) teste alfa e teste de regressão.

(E) teste de usabilidade e teste de integração.

Você grifou "desenvolvedores", "próprios escritórios" e "sem acompanhamento da equipe", que são as pistas exatas. A (B) inverte a ordem, e as demais não descrevem usuários testando antes do lançamento.

📌 **Alfa** = usuário testa no **ambiente do desenvolvedor**, observado · **beta** = usuário testa no **próprio ambiente**, sem a equipe.

---

## Questão 3 · ⬜ Não sabia (?)

O sistema de peticionamento eletrônico de um Tribunal de Justiça aceita a juntada de documentos com quantidade de páginas entre 1 e 300, inclusive. A quantidade de páginas é sempre um número inteiro.

Aplicando-se a técnica de análise de valor-limite, com dois valores por fronteira, o conjunto de entradas a ser testado é:

(A) 1, 150 e 300.

(B) 0, 150 e 301.

(C) 1 e 300.

✅ **(D) 0, 1, 300 e 301.**

(E) 0, 1, 2, 299, 300 e 301.

**O que faltou:** você grifou "1 e 300, inclusive" e "dois valores por fronteira", mas não sabia montar o conjunto. A ideia é que **os defeitos moram nas bordas**: o programador escreve < no lugar de <=. Por isso se testa cada fronteira com o **valor da própria borda** e o **vizinho do lado de fora**:

| Fronteira | Valor da borda (válido) | Vizinho de fora (inválido) |
|---|---|---|
| Inferior (1) | 1 | 0 |
| Superior (300) | 300 | 301 |

Resultado: **0, 1, 300, 301**. A (E) é a versão com **três** valores por fronteira (inclui também o vizinho de dentro, 2 e 299). A (A) e a (B) usam 150, que é valor do **meio**: isso é partição de equivalência, não valor-limite. A (C) esquece os inválidos.

📌 **Valor-limite com 2 valores:** em cada borda, a própria borda e o vizinho de fora. Faixa [1, 300] → **0, 1, 300, 301**. Com 3 valores, soma-se o vizinho de dentro (2 e 299).

---

## Questão 4 · ⬜ Não sabia (?)

O sistema de cadastro de conciliadores de um Tribunal de Justiça aceita candidatos com idade entre 21 e 65 anos, inclusive. A idade é informada sempre como número inteiro.

Utilizando a técnica de particionamento em classes de equivalência, considerando classes válidas e inválidas e um caso de teste para cada classe, a quantidade mínima de casos de teste para o campo idade é:

(A) 2.

✅ **(B) 3.**

(C) 4.

(D) 6.

(E) 45.

**O que faltou:** a partição divide as entradas em **grupos que o sistema trata do mesmo jeito**. Se 30 anos passa, 40 também passa: testar os dois não acrescenta nada. Então basta **um representante por grupo**:

| Classe | Faixa | Tipo | Exemplo de caso |
|---|---|---|---|
| Abaixo | ≤ 20 | Inválida | 15 |
| Dentro | 21 a 65 | Válida | 40 |
| Acima | ≥ 66 | Inválida | 70 |

São **3 classes → 3 casos**. A (A) esquece uma das inválidas, a (C) é o valor-limite com 4 valores (outra técnica), e a (E) é a quantidade de idades válidas (teste exaustivo).

📌 **Partição de equivalência:** uma faixa [mín, máx] gera **3 classes** (abaixo inválida, dentro válida, acima inválida) → **1 caso por classe**. Valor do meio = partição; valor da borda = valor-limite.

---

## Questão 5 · ⬜ Não sabia (?)

Sobre as técnicas de teste de software, analise as afirmativas a seguir.

I. Na técnica de caixa-branca, os casos de teste são derivados da estrutura interna do código, como na busca por cobertura de comandos e de decisões. → **V**

II. A análise de valor-limite e a tabela de decisão são técnicas de caixa-branca, pois exigem que o testador conheça as condições escritas no código-fonte. → **F**

III. Na técnica de caixa-preta, o testador pode projetar os casos de teste a partir da especificação, sem acesso ao código-fonte. → **V**

Está correto o que se afirma em

(A) I, apenas.

(B) II, apenas.

(C) I e II, apenas.

(D) II e III, apenas.

✅ **(E) I e III, apenas.**

**O que faltou:** o nome já dá a pista. Na **caixa-branca** (ou de vidro) você **vê o código por dentro**, então o teste nasce da estrutura: linhas, `if`, caminhos. Na **caixa-preta** você **não vê o código**, só a especificação ("aceita de 1 a 300 páginas"). O item II erra porque valor-limite e tabela de decisão saem **da regra de negócio** (a especificação), não do código. Você usou valor-limite na Q3 sem precisar ver uma linha de código.

📌 **Caixa-branca** = estrutura do código (cobertura de comandos/decisões, complexidade ciclomática) · **caixa-preta** = especificação, sem ver o código (valor-limite, partição, tabela de decisão, transição de estados).

---

## Questão 6 · ❌ Errou

Considere a função a seguir, escrita em Python, que calcula a taxa de expedição de uma certidão:

```
def taxa_certidao(valor, gratuidade):
    taxa = valor * 0.01
    if gratuidade:
        taxa = 0
    return taxa
```

A equipe de testes executou um único caso de teste: `taxa_certidao(1000, True)`.

Com esse único caso, é correto afirmar que foi obtida

❌ **(A) 100% de cobertura de comandos e 100% de cobertura de decisões.** ← sua resposta

✅ **(B) 100% de cobertura de comandos e 50% de cobertura de decisões.**

(C) 75% de cobertura de comandos e 50% de cobertura de decisões.

(D) 50% de cobertura de comandos e 100% de cobertura de decisões.

(E) 100% de cobertura de decisões, o que garante 100% de cobertura de caminhos.

**Onde você se confundiu:** a primeira metade você acertou. Com `gratuidade = True`, **todas as 4 linhas rodam**, então são 100% de comandos. O erro foi achar que "rodou tudo" também cobre as decisões. A decisão `if gratuidade` tem **dois lados**, o True e o False, e o caso testou **só o True**. O lado False (pular o `taxa = 0`) nunca foi exercitado. Um lado de dois = **50%**.

Rastreando:

| Linha | Executou com (1000, True)? |
|---|---|
| `taxa = valor * 0.01` | Sim |
| `if gratuidade:` | Sim (saiu pelo **True**) |
| `taxa = 0` | Sim |
| `return taxa` | Sim |
| Decisão: lado True / lado False | True ✅ · False ❌ → **50%** |

Para chegar a 100% de decisões, falta um segundo caso: `taxa_certidao(1000, False)`.

📌 **Cobertura de decisões** exige que cada `if` saia **pelo True e pelo False**. Um `if` sem `else` deixa ter 100% de comandos com só 50% de decisões. Decisões 100% ⇒ comandos 100%, **nunca o contrário**, e não garante caminhos (por isso a E está errada).

---

## Questão 7 · ✅ Acertou

Considere a função a seguir, escrita em Python, que classifica a situação de um processo:

```
def classificar(dias, urgente):
    if urgente:
        return "PRIORITARIO"
    if dias > 100:
        return "ATRASADO"
    elif dias > 30:
        return "ATENCAO"
    return "NO PRAZO"
```

A complexidade ciclomática dessa função, que indica o número mínimo de caminhos independentes a serem testados, é igual a:

(A) 2.

(B) 3.

✅ **(C) 4.** ← sua resposta

(D) 5.

(E) 6.

Ótimo: você desenhou as ramificações ao lado do código e contou 4 saídas. Pela fórmula rápida: 3 decisões (`if urgente`, `if dias > 100`, `elif dias > 30`) + 1 = **4**. Cada caminho termina num `return` diferente.

📌 **Complexidade ciclomática = decisões + 1** (cada `if`, `elif`, `while` e `for` conta 1). É o mínimo de casos para cobrir os caminhos independentes.

## Questão 8 · ✅ Acertou

No último dia de um prazo processual de grande repercussão, um Tribunal de Justiça estima que até 8.000 advogados acessem simultaneamente o sistema de peticionamento. Antes dessa data, a equipe de TI realizou dois testes:

- no primeiro, simulou 8.000 usuários simultâneos, para verificar se o tempo de resposta se mantinha dentro do limite definido nos requisitos;
- no segundo, aumentou progressivamente a quantidade de usuários simulados até 30.000, bem acima da capacidade prevista, para observar como o sistema falha e se ele se recupera depois.

Os dois testes descritos são, respectivamente, de:

✅ **(A) carga e estresse.** ← sua resposta

(B) estresse e carga.

(C) regressão e volume.

(D) aceitação e pico.

(E) usabilidade e resistência.

Você grifou "8.000 usuários simultâneos", "tempo de resposta", "até 30.000" e "como o sistema falha e se recupera": exatamente as pistas. A (B) inverte a ordem.

📌 **Carga** = volume **esperado** (cumpre o requisito?) · **estresse** = **além da capacidade** (como falha e se recupera).

## Questão 9 · ❌ Errou

Um defeito no cálculo de prazos em dobro para a Defensoria Pública foi corrigido no sistema processual de um Tribunal de Justiça. Em seguida, a equipe de testes:

- executou novamente o caso de teste que havia revelado o defeito, para confirmar que ele foi de fato corrigido;
- reexecutou a suíte automatizada das demais funcionalidades (intimações, distribuição e certidões), para verificar se a alteração não introduziu defeitos no que já funcionava.

As duas atividades descritas são, respectivamente, teste de:

❌ **(A) regressão e teste de confirmação.** ← sua resposta

(B) unidade e teste de integração.

(C) fumaça (smoke) e teste de sanidade.

(D) aceitação e teste de regressão.

✅ **(E) confirmação (reteste) e teste de regressão.**

**Onde você se confundiu:** você ligou a **primeira** atividade a "regressão" (a linha que você traçou vai do primeiro item até a A). A palavra "executou **novamente**" puxou para regressão, porque regressão também é "rodar de novo". O que separa os dois é **o que** se roda de novo:

| | O que roda de novo | Pergunta que responde |
|---|---|---|
| **Confirmação (reteste)** | **O mesmo caso** que achou o defeito | "Aquele bug foi corrigido?" |
| **Regressão** | **O resto** do sistema, que já funcionava | "A correção quebrou outra coisa?" |

A ordem natural é essa: primeiro se confirma a correção, depois se roda a regressão. A pista do 2º item era "no que **já funcionava**", que é a definição de regressão.

📌 **Confirmação** = o mesmo caso que falhou, para ver se o defeito foi corrigido · **regressão** = as **outras** funções, para ver se a mudança quebrou o que funcionava. "Já funcionava" = regressão.

---

## Questão 10 · ✅ Acertou

Ao especificar o cálculo de prazos de um sistema processual, um analista entendeu equivocadamente que o prazo deveria ser contado em dias corridos, e não em dias úteis. Com base nesse entendimento, o desenvolvedor implementou a contagem em dias corridos no código-fonte. Após a implantação, um advogado consultou o sistema e viu, na tela, uma data de vencimento anterior à correta.

Segundo a terminologia de teste de software, o entendimento equivocado do analista, a contagem incorreta no código-fonte e a data errada exibida ao advogado são, respectivamente:

(A) falha, defeito e erro.

(B) defeito, erro e falha.

(C) erro, falha e defeito.

✅ **(D) erro, defeito e falha.** ← sua resposta

(E) defeito, falha e erro.

Você riscou A, B e E e ficou com a ordem certa: começa na **pessoa**, passa pelo **código** e termina no que **aparece na execução**.

📌 **Erro** (engano humano) → **defeito** (no código ou no documento) → **falha** (comportamento errado observado na execução).

---

## Questão 11 · ✅ Acertou

Uma equipe adota a integração incremental descendente (top-down) no sistema de custas de um Tribunal de Justiça. O módulo de distribuição, no topo da hierarquia, já está pronto, mas o módulo de cálculo de custas, que é chamado por ele, ainda não foi implementado.

Para testar o módulo de distribuição, a equipe deve criar um

(A) driver, que simula o módulo de cálculo de custas e devolve respostas predefinidas ao módulo de distribuição.

✅ **(B) stub, que simula o módulo de cálculo de custas, devolvendo respostas predefinidas quando chamado pelo módulo de distribuição.** ← sua resposta

(C) driver, que simula um módulo superior responsável por chamar o módulo de distribuição.

(D) stub, que chama o módulo de distribuição e verifica as respostas por ele produzidas.

(E) oráculo de teste, que substitui definitivamente o módulo de cálculo de custas em produção.

Você grifou "módulo de cálculo de custas, que é chamado" e riscou os drivers. O que falta é o módulo **de baixo** (o chamado), então a peça é o stub. A (D) dá ao stub o papel de driver (quem chama).

📌 **Top-down usa stub** (substitui quem é **chamado**) · **bottom-up usa driver** (substitui quem **chama**).

---

## Questão 12 · ❌ Errou

Sobre os princípios do teste de software, assinale a afirmativa **incorreta**.

❌ **(A) O teste pode demonstrar a presença de defeitos, mas não pode provar a sua ausência.** ← sua resposta

(B) O teste exaustivo, com todas as combinações de entradas e pré-condições, é inviável, exceto em casos triviais; por isso, usam-se análise de risco e técnicas de teste para priorizar.

(C) Repetir sempre os mesmos testes tende a fazer com que eles deixem de encontrar defeitos novos, sendo necessário revisá-los periodicamente.

✅ **(D) Um sistema no qual não foram encontrados defeitos após testes extensivos certamente atende às necessidades e expectativas dos usuários.**

(E) Os defeitos tendem a se concentrar em um pequeno número de módulos do sistema.

**Onde você se confundiu:** a estratégia do V/F funcionou em parte, porque você deu ✓ em (B) e (C). Mas a (A), que você circulou, é o **1º princípio do teste** e está **certa**. Provavelmente o "não pode provar a sua ausência" soou como uma limitação "negativa" e, por isso, falsa. A falsa era a (D): ela diz que "sem defeitos" garante que o sistema atende ao usuário. Isso é a **falácia da ausência de erros**: o sistema pode estar sem bugs e mesmo assim fazer a coisa errada (lembre de verificação × validação). A palavra "**certamente**" era a pista: em princípio de teste, certeza absoluta costuma ser a alternativa errada.

Os 7 princípios (ISTQB), para conferir as outras:

| Alternativa | Princípio | V/F |
|---|---|---|
| A | Teste mostra a presença, não a ausência | V |
| B | Teste exaustivo é impossível | V |
| C | Paradoxo do pesticida | V |
| D | Falácia da ausência de erros (escrita ao contrário) | **F** |
| E | Agrupamento de defeitos | V |

📌 **Teste mostra a presença de defeitos, nunca prova a ausência** (é verdade e cai muito). **"Sem defeitos" não quer dizer "atende ao usuário"** (falácia da ausência de erros). Em "incorreta", desconfie de "certamente", "sempre" e "garante".

---

## Questão 13 · ⬜ Não sabia (?)

A Secretaria de TI de um Tribunal de Justiça realizou as seguintes atividades de garantia da qualidade em um novo sistema:

I. inspeção do documento de requisitos por analistas e usuários-chave; → **estático**

II. análise do código-fonte por uma ferramenta que aponta variáveis não utilizadas e trechos duplicados; → **estático**

III. execução da suíte de testes unitários, com medição de cobertura; → **dinâmico**

IV. revisão do código pelos pares antes da aprovação de um pull request. → **estático**

São exemplos de teste estático as atividades

(A) I e III, apenas.

(B) II e III, apenas.

(C) III e IV, apenas.

(D) I, II, III e IV.

✅ **(E) I, II e IV, apenas.**

**O que faltou:** você circulou "teste estático" e pôs o "?" ali, então o conceito é que faltou. O critério é um só: **o software está rodando?**

| | O software roda? | Exemplos |
|---|---|---|
| **Estático** | **Não**: analisa documento ou código parado | Inspeção de requisitos, revisão de código (PR), análise estática por ferramenta (SonarQube, que é a base do SAST de sexta) |
| **Dinâmico** | **Sim**: executa e observa o resultado | Testes unitários, integração, carga, regressão |

Só o III **executa** o código (rodar a suíte). Os outros três olham para algo parado, inclusive o II, que usa ferramenta mas **não executa** o programa.

📌 **Teste estático = sem executar**: inspeção de documento, revisão de código e análise estática por ferramenta. **Executou = dinâmico**, mesmo que seja automatizado.

---

## Questão 14 · ✅ Acertou

Um Tribunal de Justiça está adotando integração contínua e automação de testes no desenvolvimento do seu sistema processual.

Sobre a automação de testes e a pirâmide de testes, assinale a afirmativa correta.

✅ **(A) A base da pirâmide deve concentrar a maior quantidade de testes, os unitários, por serem rápidos e baratos; os testes de interface ponta a ponta ficam em menor número no topo, por serem lentos e frágeis.** ← sua resposta

(B) A maior parte da suíte deve ser composta por testes de interface ponta a ponta, por reproduzirem fielmente o uso real e serem mais estáveis que os testes unitários.

(C) Os testes de regressão são maus candidatos à automação, pois precisam ser reescritos a cada nova versão do sistema.

(D) Com a automação, os testes manuais e exploratórios tornam-se dispensáveis, pois a suíte automatizada cobre todos os cenários possíveis.

(E) Na integração contínua, os testes automatizados devem ser executados apenas antes da implantação em produção, para não atrasar os commits da equipe.

A pirâmide do vídeo ficou. A (B) inverte a pirâmide; a (C) erra porque a regressão é a **melhor** candidata à automação (repete sempre); a (D) erra porque o exploratório continua necessário; a (E) erra porque na CI os testes rodam **a cada commit**.

📌 **Pirâmide:** muitos unitários na base, poucos E2E no topo. **Regressão** é a melhor candidata à automação. Na **CI**, os testes rodam a cada commit. Automação não substitui o exploratório.

---

## Questão 15 · ✅ Acertou

Uma equipe que pratica TDD está desenvolvendo a classe `CalculadoraPrazo`, que soma dias úteis a uma data, ignorando sábados, domingos e feriados. A consulta de feriados é feita por um serviço externo do Tribunal. Antes de implementar a classe, a equipe escreveu o teste a seguir, em Python:

```
class FeriadosStub:
    def eh_feriado(self, data):
        return data in {"2026-11-20"}

def test_prazo_pula_feriado():
    calc = CalculadoraPrazo(FeriadosStub())
    # 19/11/2026 é quinta-feira; 20/11 é feriado
    assert calc.vencimento("2026-11-19", 1) == "2026-11-23"
```

Sobre esse teste, é correto afirmar que

(A) se trata de um teste de integração, pois a `CalculadoraPrazo` interage com outra classe durante a execução.

(B) `FeriadosStub` é um mock, pois verifica quantas vezes o método `eh_feriado` foi chamado e com quais parâmetros.

✅ **(C) `FeriadosStub` isola a unidade do serviço real de feriados, fornecendo respostas predefinidas, o que torna o teste unitário rápido e determinístico.** ← sua resposta

(D) pelo TDD, o teste só poderia ter sido escrito depois de implementada a `CalculadoraPrazo`, para que o primeiro resultado da execução fosse verde.

(E) como o `assert` apenas compara datas, trata-se de um teste estático, que não exige a execução do código.

Você riscou a (A) e deixou pontinho na (B) antes de fechar na (C). A pista contra a (B) está no código: o stub só **devolve** `True`/`False` e não guarda nenhuma contagem de chamadas. Quem verifica chamadas é o mock. A (D) inverte o TDD (o teste vem antes e começa vermelho), e a (E) erra porque rodar o `assert` é executar o código (dinâmico).

📌 **Stub** = devolve respostas prontas (verifica-se o estado) · **mock** = verifica as chamadas (comportamento). Usar dublê **mantém o teste unitário**. No TDD, o teste vem antes da classe e o primeiro resultado é vermelho.

---

## Para o caderno de erros

| Assunto | Por que errei | Regra certa |
|---|---|---|
| Valor-limite (Q3, ?) | Não sabia montar o conjunto | Em cada borda: a borda e o vizinho de fora. [1, 300] → 0, 1, 300, 301 |
| Partição de equivalência (Q4, ?) | Não sabia contar as classes | Faixa = 3 classes (abaixo, dentro, acima) → 3 casos |
| Caixa-branca × preta (Q5, ?) | Não sabia de onde vem cada técnica | Branca = código (cobertura) · preta = especificação (valor-limite, partição, tabela de decisão) |
| Cobertura de decisões (Q6) | "Rodou tudo" = 100% de decisões | Decisão exige True **e** False. Um caso com `if` sem `else` = 100% comandos, 50% decisões |
| Confirmação × regressão (Q9) | "Executou novamente" me levou a regressão | Mesmo caso = confirmação · "o que já funcionava" = regressão |
| Princípios, "incorreta" (Q12) | Marquei como falso o princípio verdadeiro da presença × ausência | Presença sim, ausência nunca: é V. "Sem defeitos = atende ao usuário" é a falácia (F) |
| Estático × dinâmico (Q13, ?) | Não sabia o conceito | Sem executar = estático (inspeção, revisão, análise por ferramenta). Executou = dinâmico |

---

## O que estudar agora

| Quando | O que fazer |
|---|---|
| Hoje · 22:40 (revisão) | Reler as 7 regras 📌 dos erros e "?". Refazer **sem olhar**: Q3 com a faixa [18, 70] (resposta: 17, 18, 70, 71), Q4 com a mesma faixa (3 classes) e Q6 escrevendo o 2º caso que leva a 100% de decisões. Anki dos cartões de hoje |
| Qui 05/11 · almoço | Próximo item de Português da fila (B1 + B2) |
| Qui 05/11 · noite | **SAFe** (item 6). Antes, 10 min: refazer a Q9 e a Q13 de cabeça |
| Sex 06/11 · noite | **SAST × DAST × IAST e SonarQube** (item 7). Liga direto com a Q13: o SAST é teste **estático**; o DAST é **dinâmico** |
| Sáb 07/11 | Recuperação: bloco de Requisitos (item 4) → APF real (item 8). Se sobrar tempo: 5 questões FGV reais só de **técnicas de teste** (valor-limite, partição, cobertura) |
| Dom 08/11 · 13h | 1º simulado oficial |